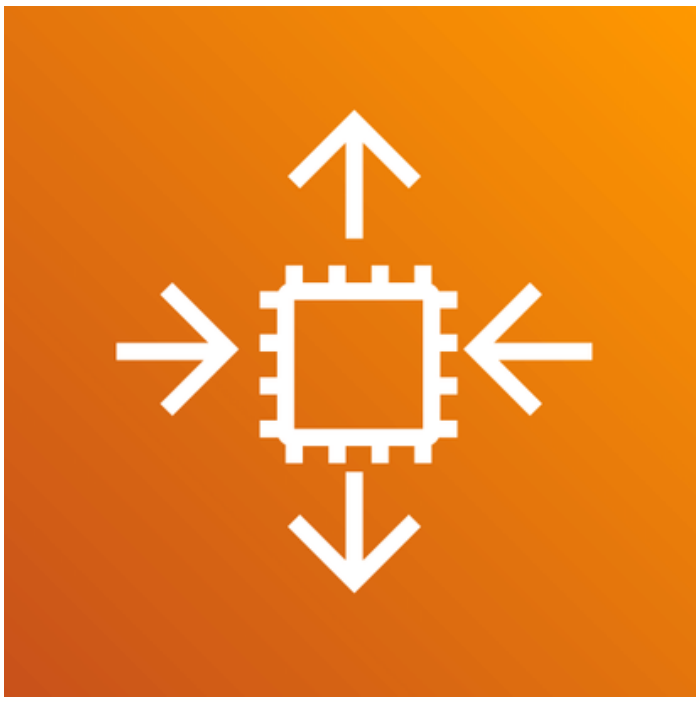


AWS ASG

Auto Scaling Groups

Concept Overview:

Understanding Scalability	1
Introduction of ASG	2
Benefits of ASG	3
Understanding Images(AMIs)	4
Understanding Launch Templates	5
Prerequisites Before Hands-on	6
Hands-on Configure ASG	7

Understanding Scalability:

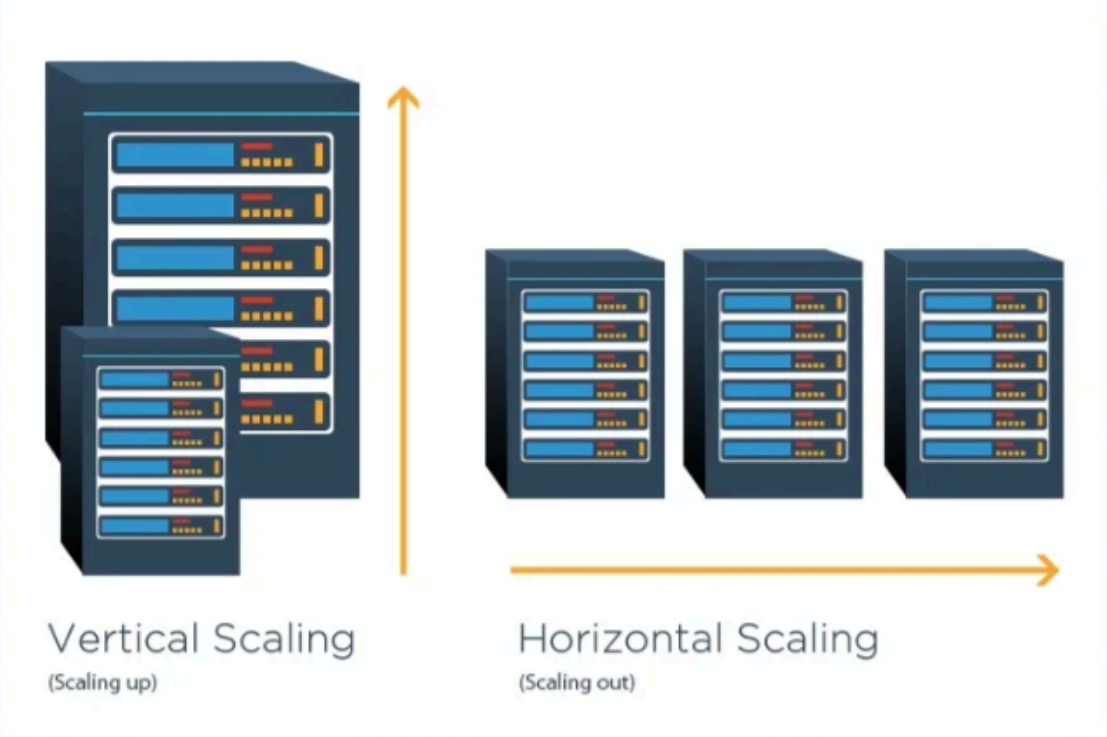
Scalability means the ability of your applications and infrastructure to handle increasing or decreasing workloads by adjusting resources automatically.

There are Two types of scalability:

1. Vertical scalability
2. Horizontal scalability

Vertical scalability: Increase the size of an instance.

Horizontal scalability: Increasing the number of instances/systems.



Introduction of ASG:

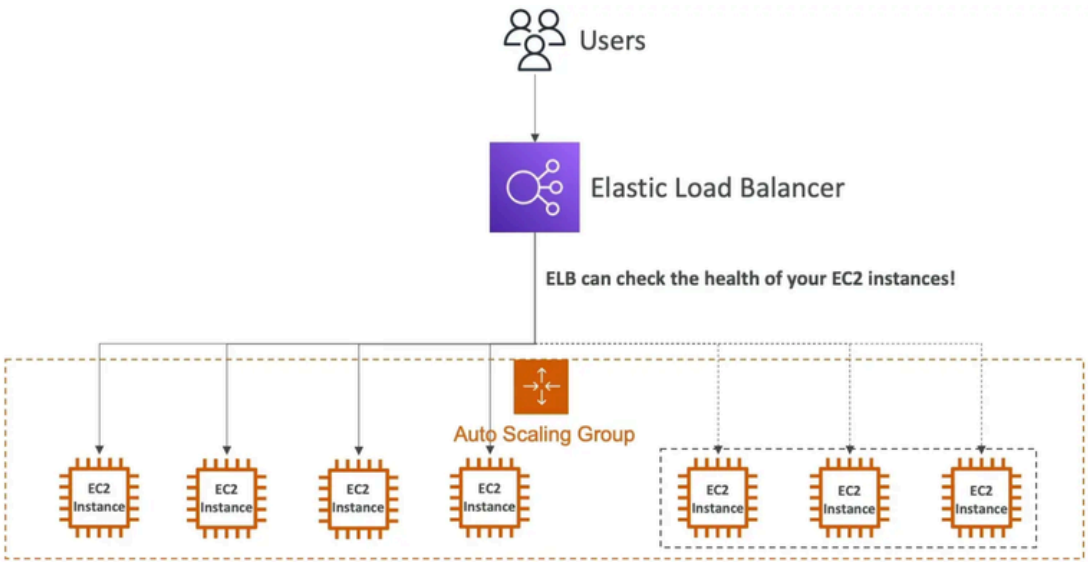
An Auto Scaling group contains a collection of EC2 instances that are treated as a logical grouping for the purposes of automatic scaling and management.

An Auto Scaling group also lets you use Amazon EC2 Auto Scaling features such as health check replacements and scaling policies.

Both maintaining the number of instances in an Auto Scaling group and automatic scaling are the core functionality of the Amazon EC2 Auto Scaling service.

The size of an Auto Scaling group depends on the number of instances that you set as the desired capacity. You can adjust its size to meet demand, either manually or by using automatic scaling.

The illustration shows ASG with ELB(Elastic Load Balancer)



Benefits of ASG:

- Automatic Scaling: Dynamically adds or removes EC2 instances based on real-time demand, ensuring optimal performance without manual intervention.

- Cost Optimization: Runs only the required number of instances, reducing unnecessary costs during low-traffic periods.

- High Availability: Distributes instances across multiple Availability Zones, improving fault tolerance and minimizing downtime.

- Health Monitoring & Self-Healing: Automatically replaces unhealthy instances to maintain application stability and performance.

- Load Handling: Ensures consistent performance during traffic spikes by quickly provisioning additional instances.

- Integration with AWS Services: Works seamlessly with Elastic Load Balancing (ELB), CloudWatch, and other AWS tools for monitoring and traffic distribution.

Understanding Images (AMIs):

An Amazon Machine Image (AMI) is an image that provides the software that is required to set up and boot an Amazon EC2 instance.

Each AMI also contains a block device mapping that specifies the block devices to attach to the instances that you launch.

You must specify an AMI when you launch an instance. The AMI must be compatible with the instance type that you chose for your instance.

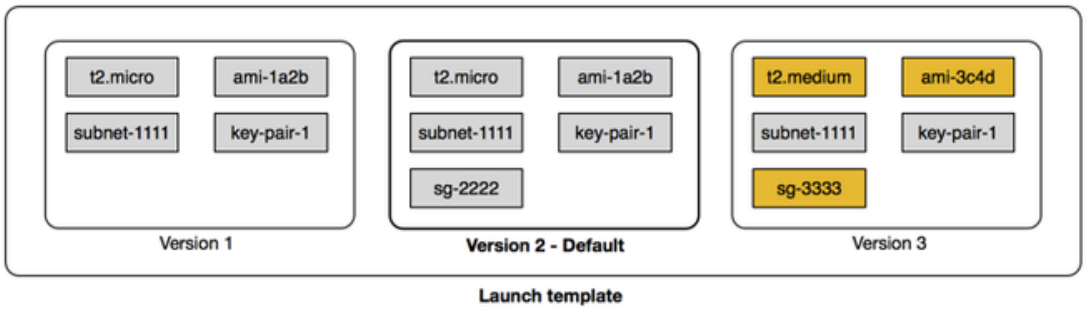
An AMI is specific to the following:

- Region
- Operating system
- Processor architecture
- Root device type
- Virtualization type

Understanding Launch Templates:

An AWS EC2 Launch Template is a reusable configuration that stores all the settings you need to launch EC2 instances consistently and easily.

You can use an Amazon EC2 launch template to store instance launch parameters so that you don't have to specify them every time you launch an Amazon EC2 instance. For example, you can create a launch template that stores the AMI ID, instance type, and network settings that you typically use to launch instances. When you launch an instance using the Amazon EC2 console, an AWS SDK, or a command line tool, you can specify the launch template instead of entering the parameters again.



Prerequisites Before Hands-on:

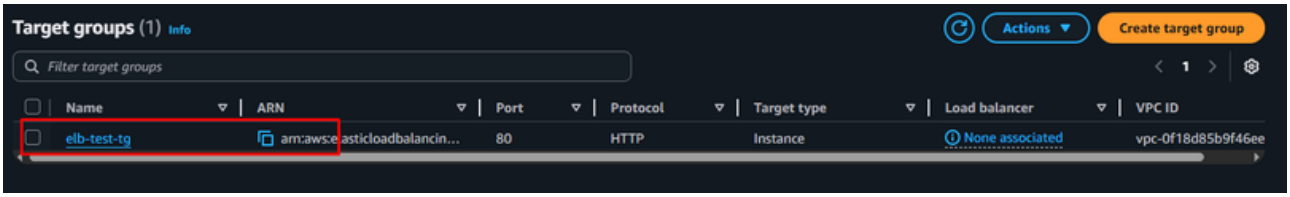
In my case, I'll use it with a load balancer. If you do the same, make sure you have the prerequisites. otherwise, you can skip this step.

Prerequisites:

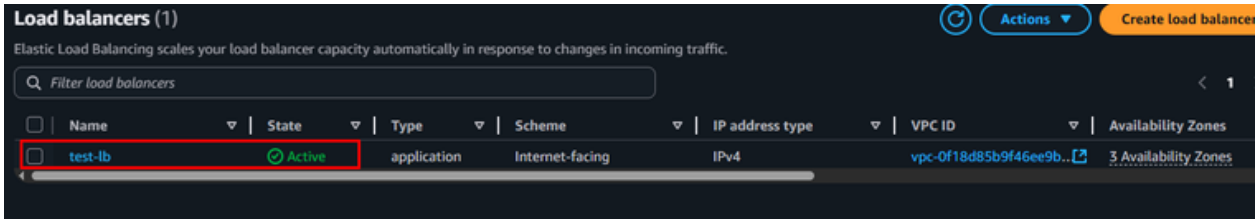
1. Launch an EC2 instance with apache/nginx server
2. Configure TG(Target Group) with EC2
3. Configure LB(Load Balancer) with TG

As covered in the previous day, I'll use it for the ASG hands-on. Here it is:

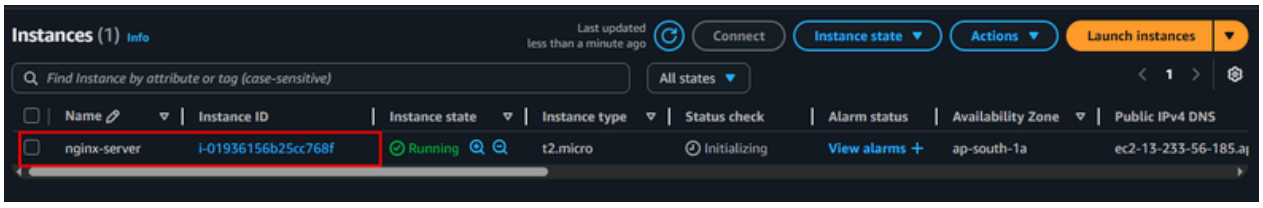
Target Group(TG)



Load balancer

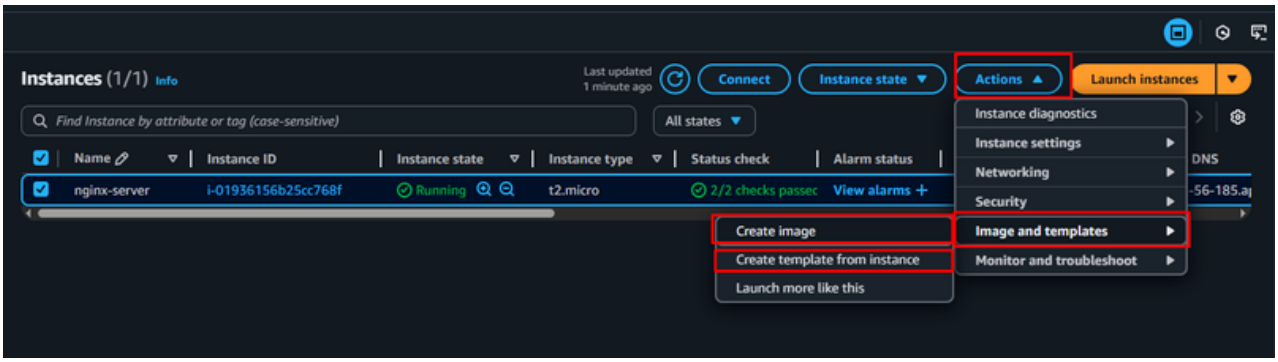


I use nginx-server EC2 instance for hands-on

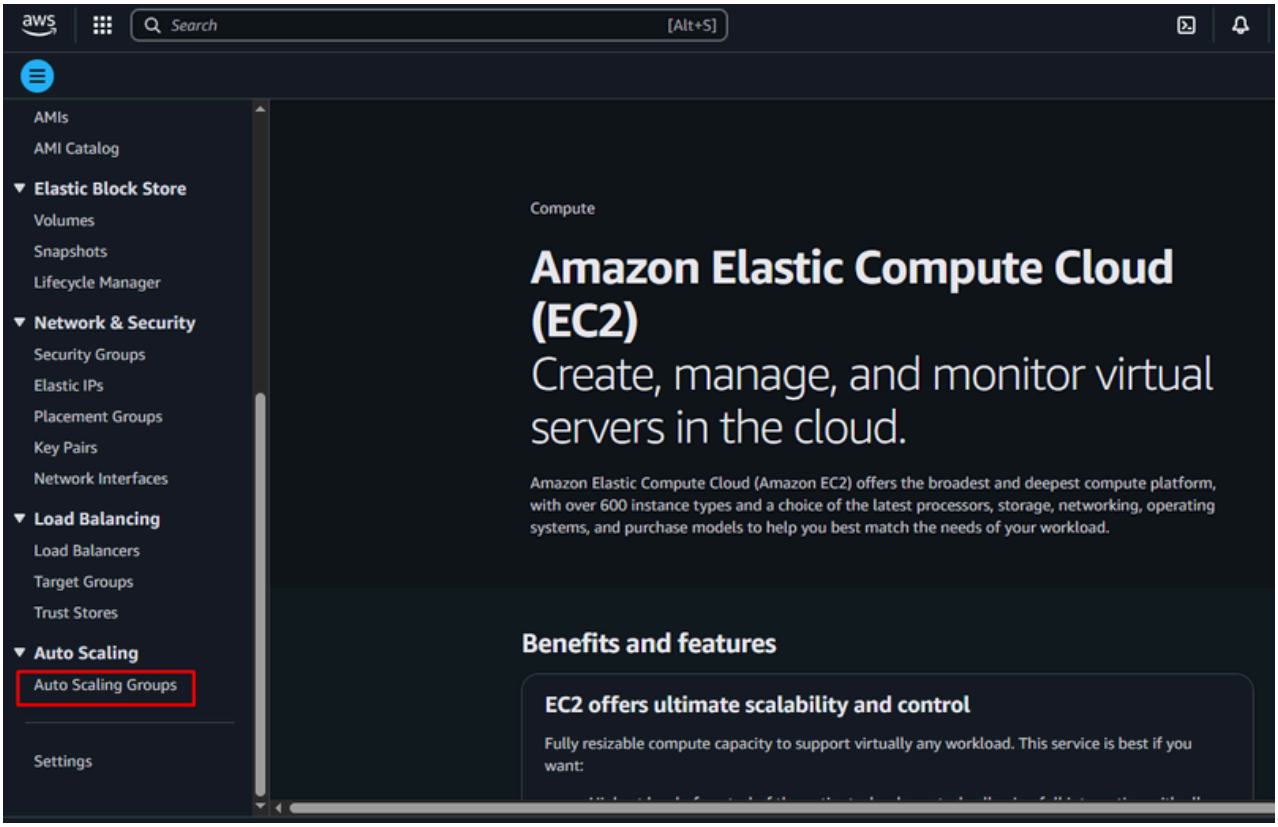


Hands-on Configure ASG:

To avoid further configuration details for Launch Templates, you can convert your instance directly into a template or image.

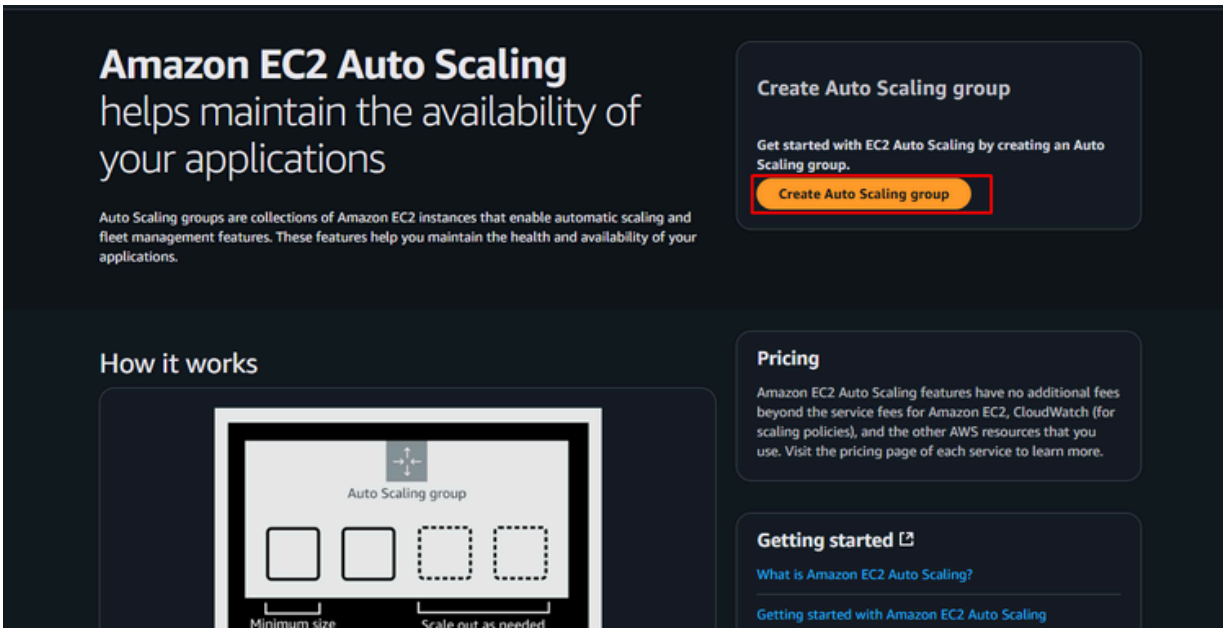


Now, configure the ASG. Go to your EC2 dashboard and click on auto scaling group.

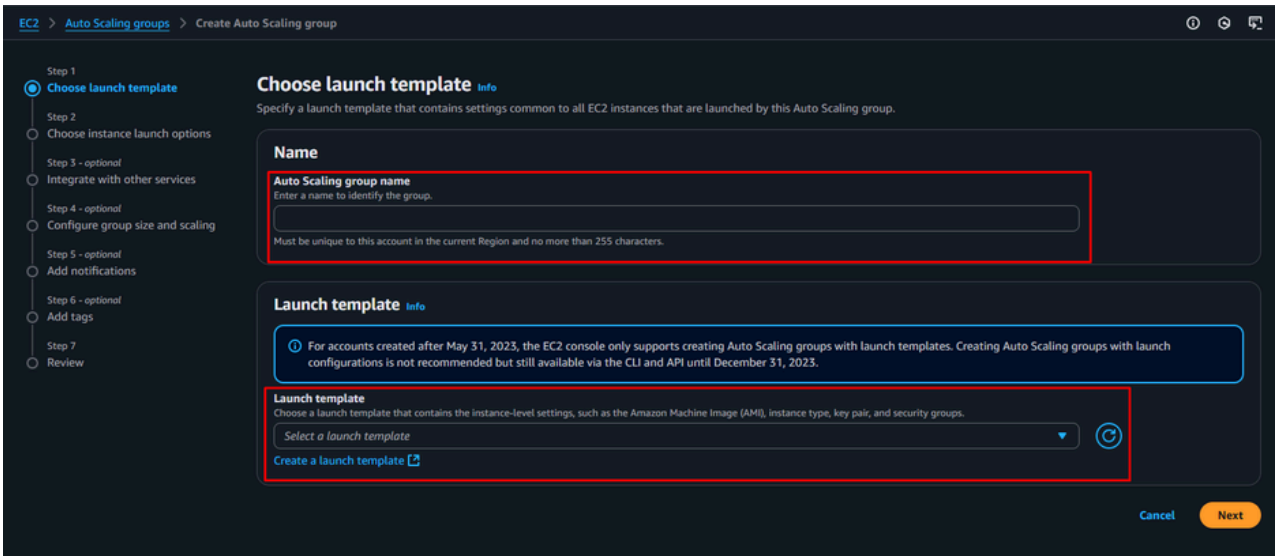


Hands-on Configure ASG:

After clicking on Auto scaling groups, get this page.



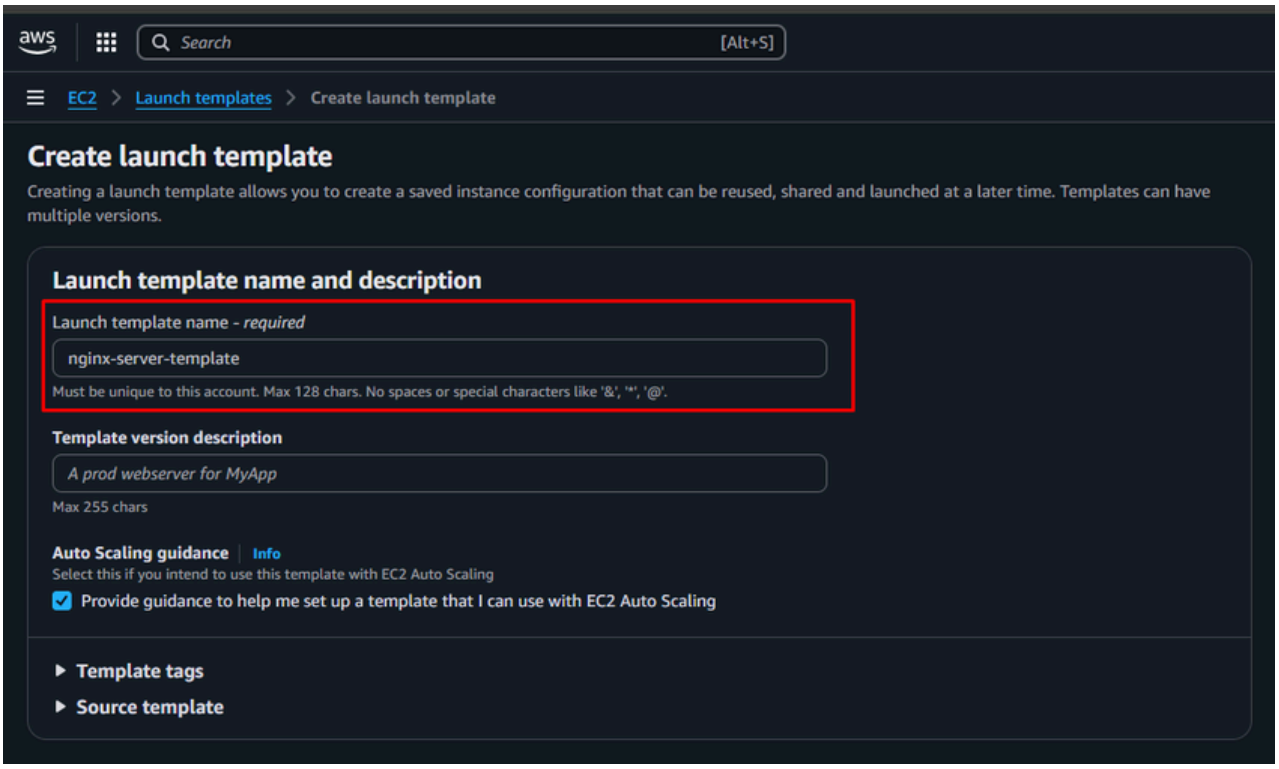
After clicking on Create Auto Scaling Group, get the configure page. Now configure based on you needs.



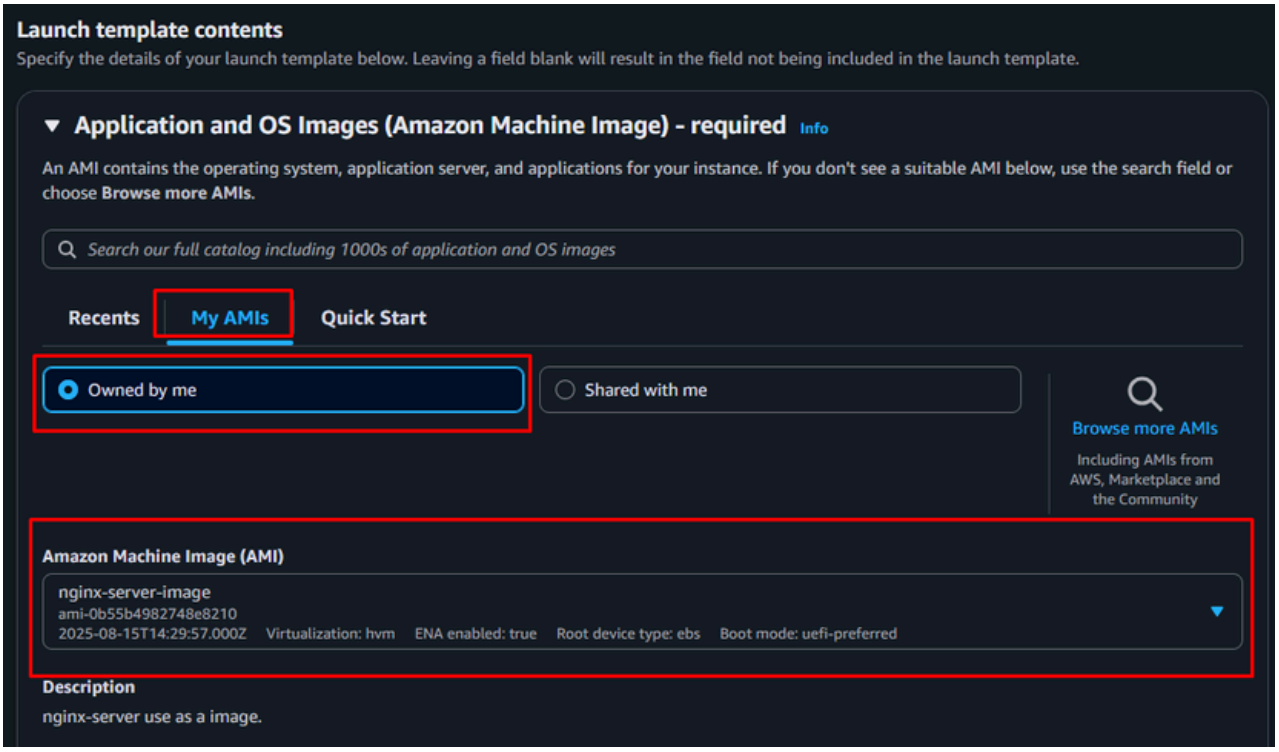
If any Lanuch template, you can choose it otherwise create new one.

Hands-on Configure ASG:

In my case, I Create a new template with existing Image which i make by my EC2 instance.

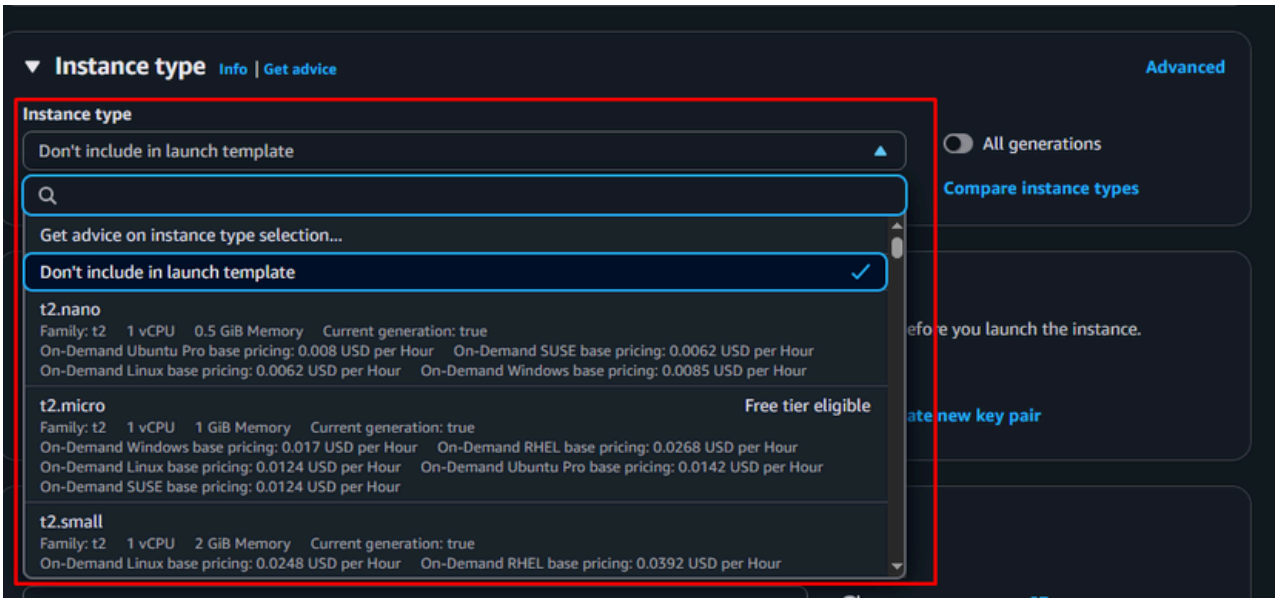


If you want to use a previously created Image, choose these options.

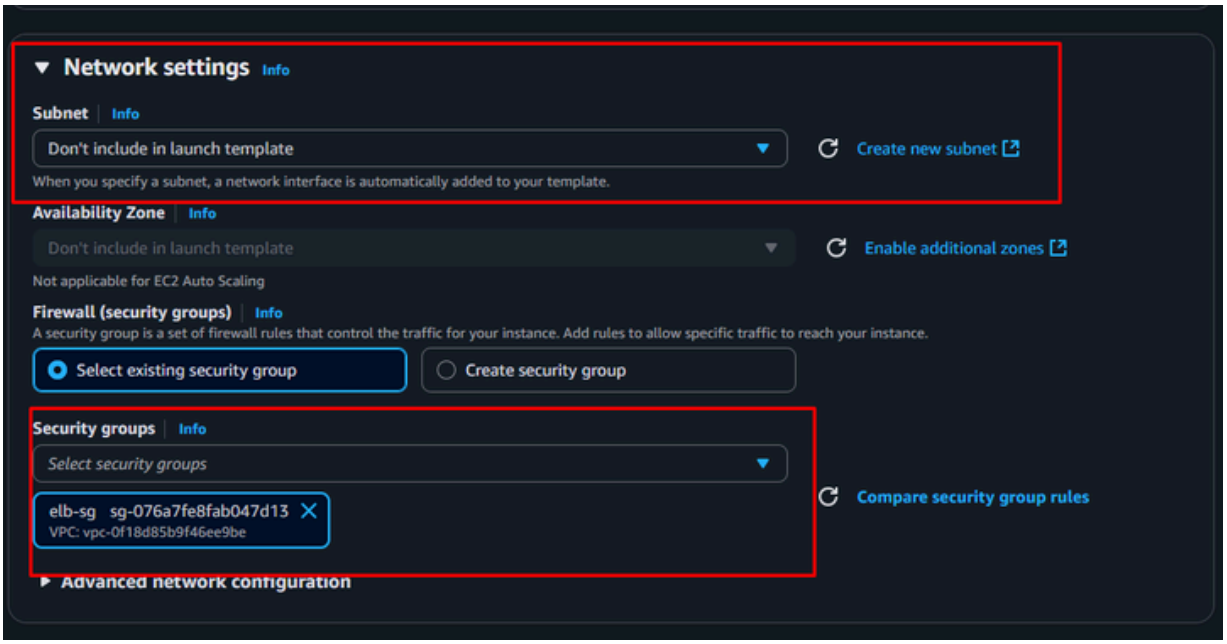


Hands-on Configure ASG:

Then choose an instance, in my case i use t2.micro free tier version😀

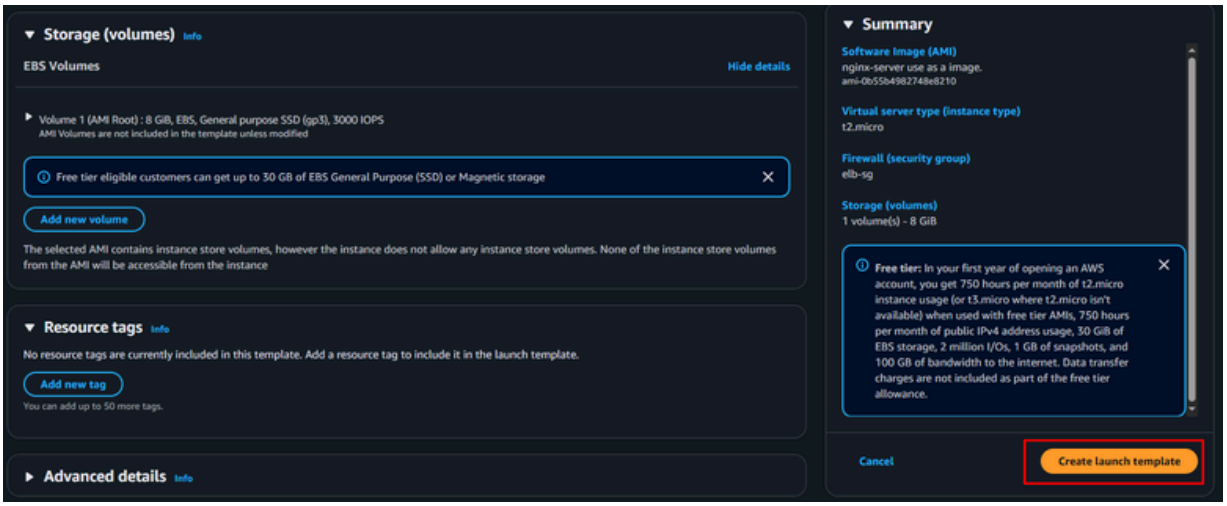


In the network settings section, select the Security Group(SG) used by your Load Balancer(LB). If you don’t have an LB, keep the default or create one.

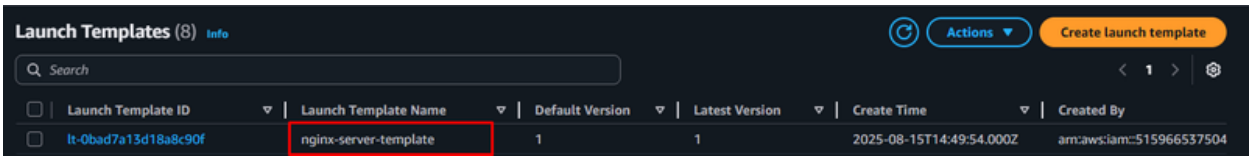


Hands-on Configure ASG:

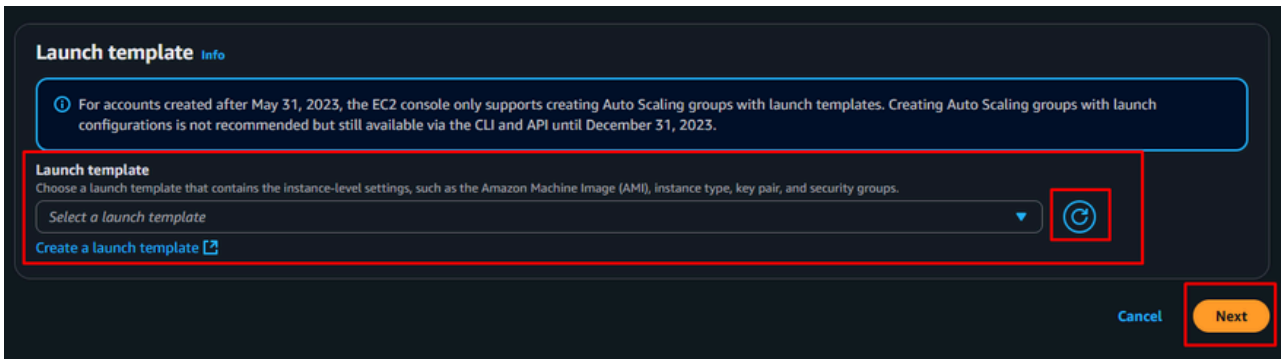
If needed, you can adjust any other configurations based on your requirements. Otherwise, keep them default and click Create Launch Template.



Launch template succefully created.

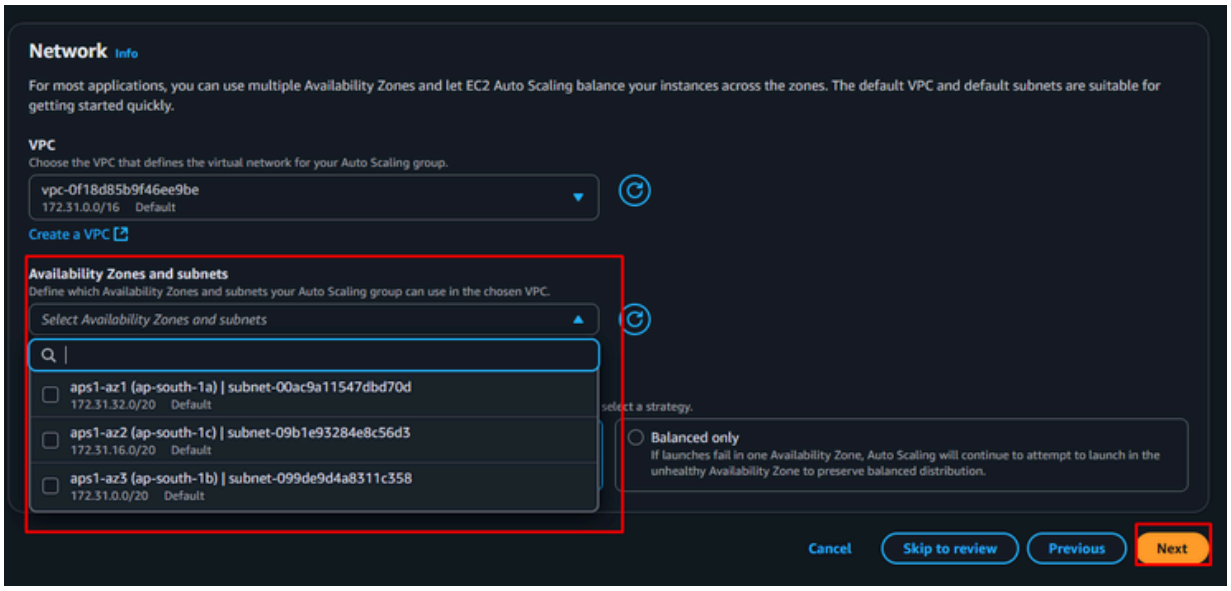


Now, refresh lanuch template section and select the template and click next button for next step.

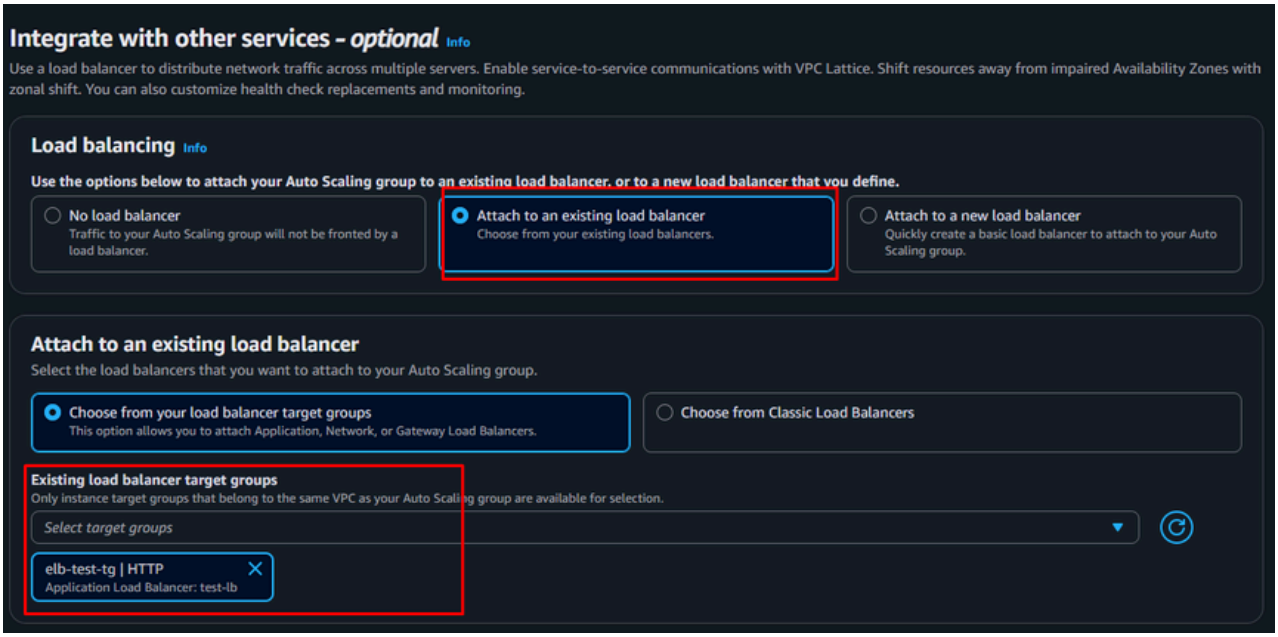


Hands-on Configure ASG:

In step 2, network part select AZ(availability zone) & VPC if you have any custom then click on next.



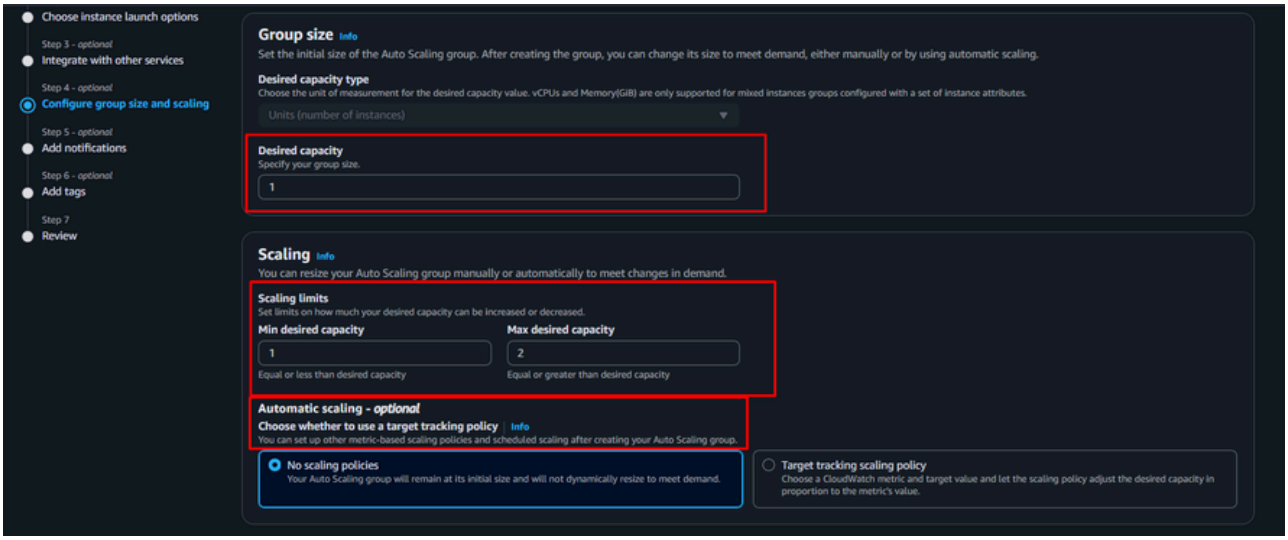
In step 3, You can attach your LB or ignore it. If don't want use. If you attach LB you need to select TG, don't forget to select TG.



Hands-on Configure ASG:

Step 4 is an important part where you can configure the desired capacity, minimum capacity, and maximum capacity.

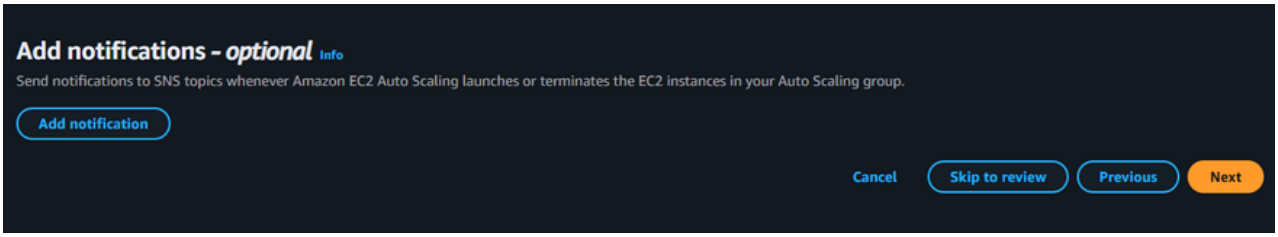
- Desired Capacity: The exact number of EC2 instances the Auto Scaling Group tries to maintain at all times.
- Minimum Capacity: The lowest number of EC2 instances that must always be running in the group.
- Maximum Capacity: The highest number of EC2 instances the group can scale up to.

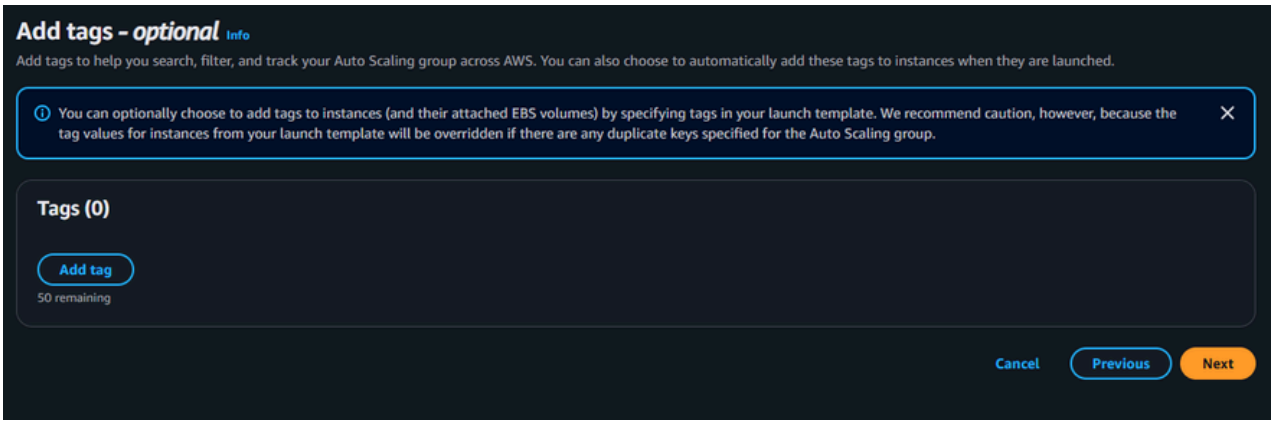


If you want tracking, you can choose Target cracking scaling policy.

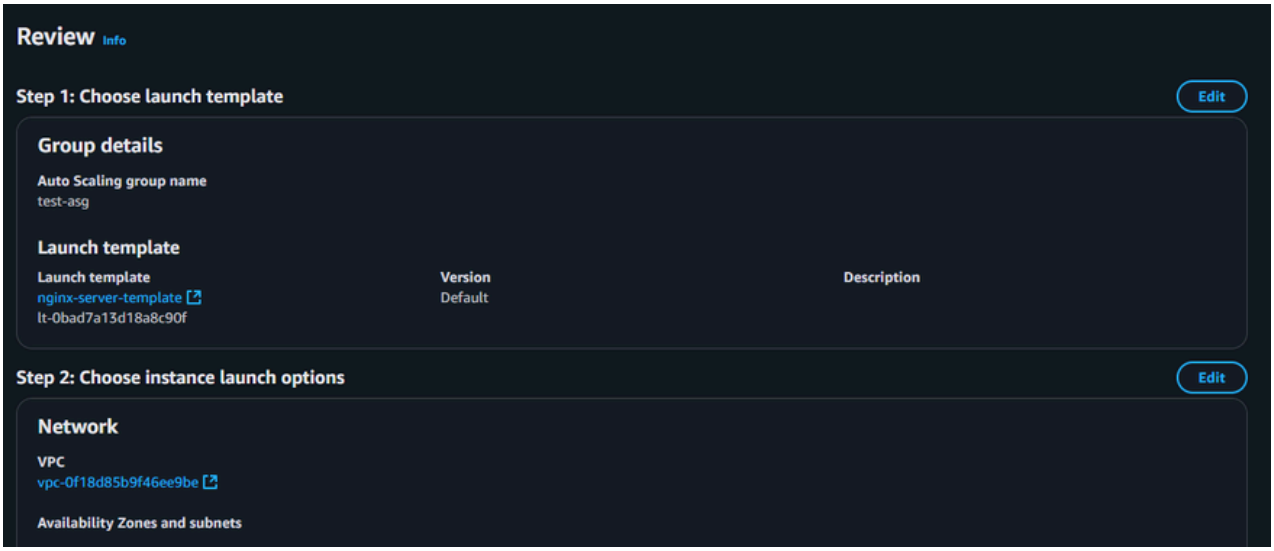
Hands-on Configure ASG:

Steps 5 & 6 let you set up notifications and tags, which you can configure if required.

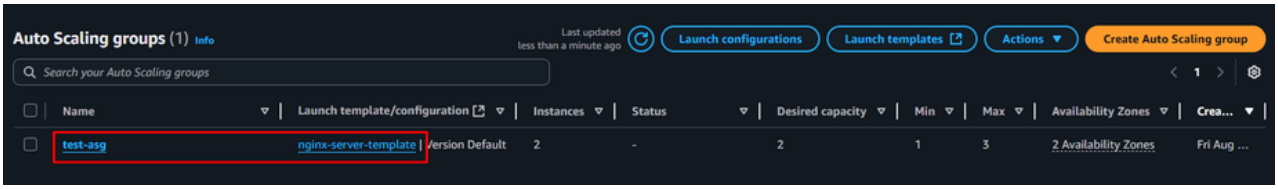




In final step, review and save it.

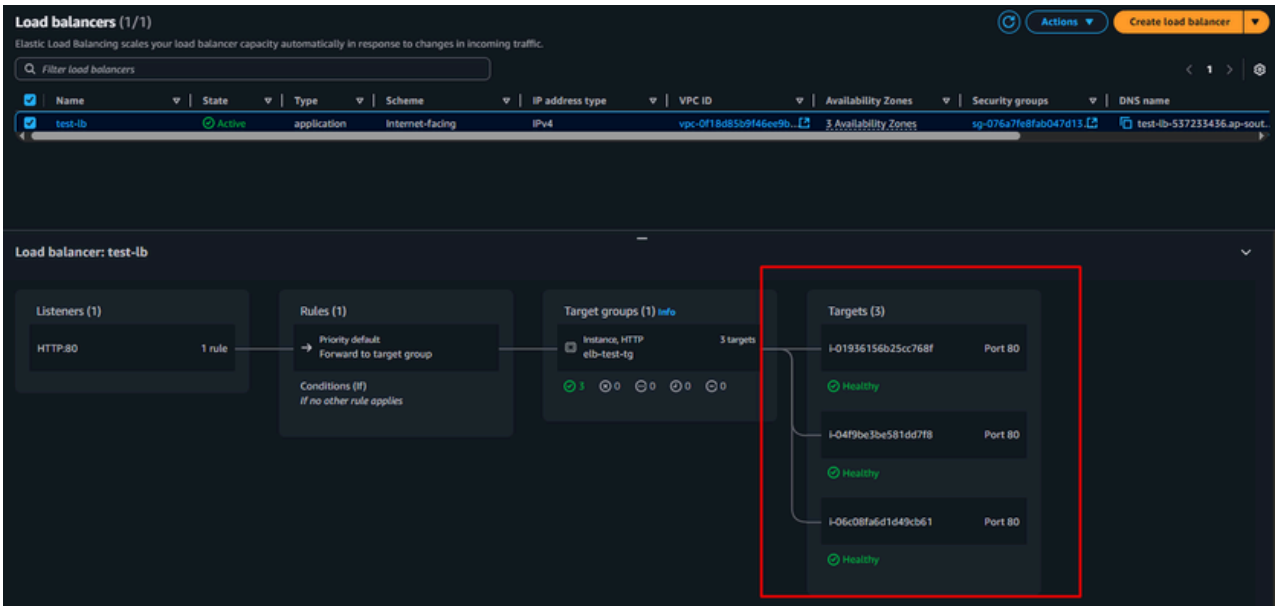


It’s created successfully.



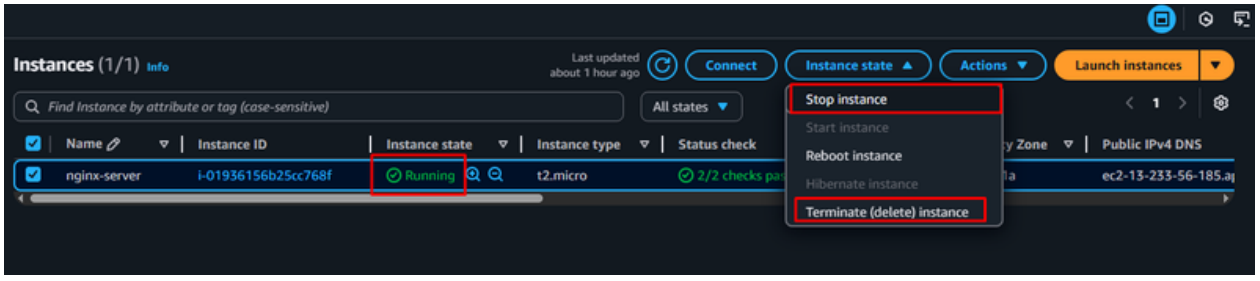
Hands-on Configure ASG:

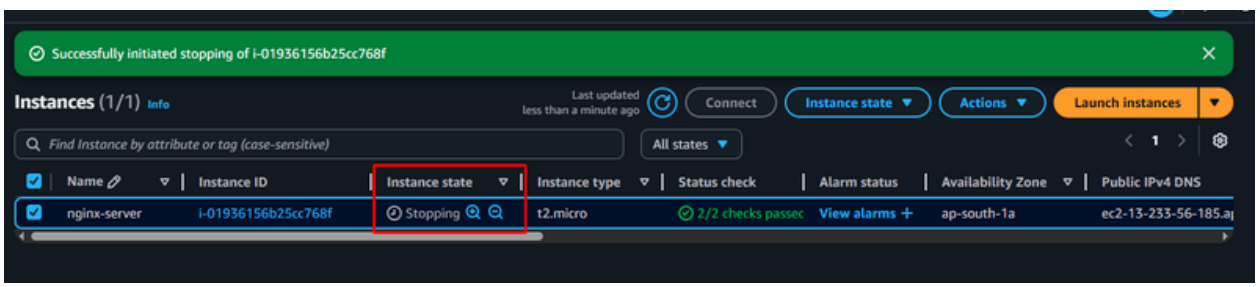
Now go to the Load Balancer section and see what happened.



We configured one instance and set the desired capacity to two, which is why it shows three instances. This means our ASG worked perfectly.

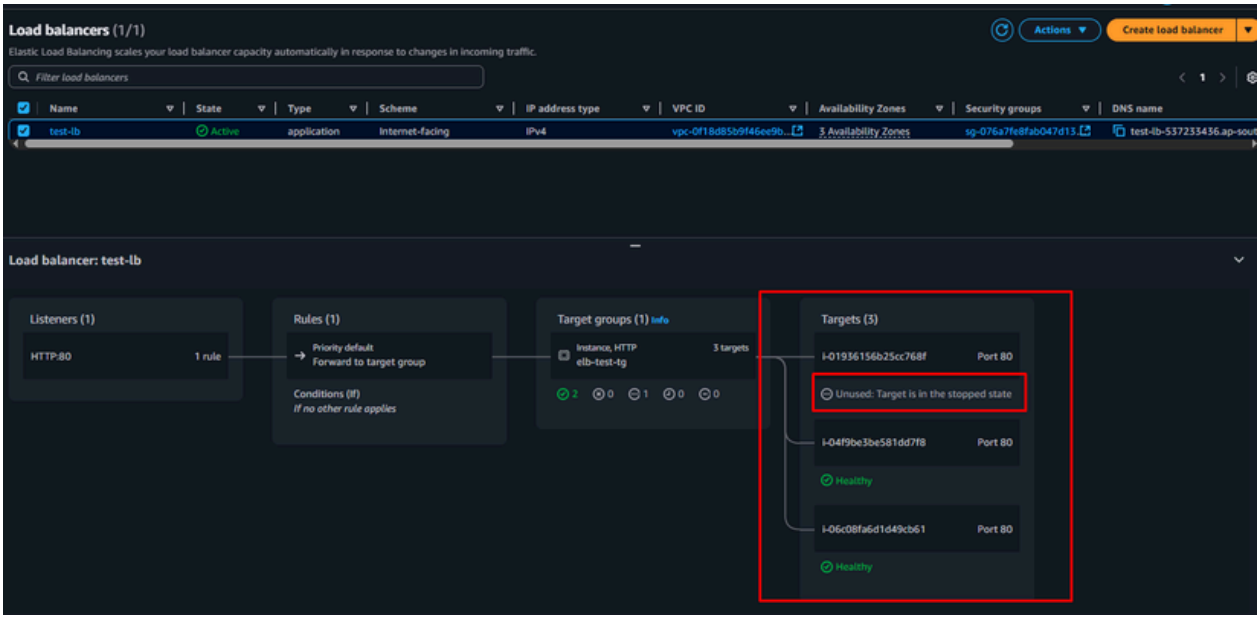
Now stop or terminate our base EC2 instance, and then see what happens.



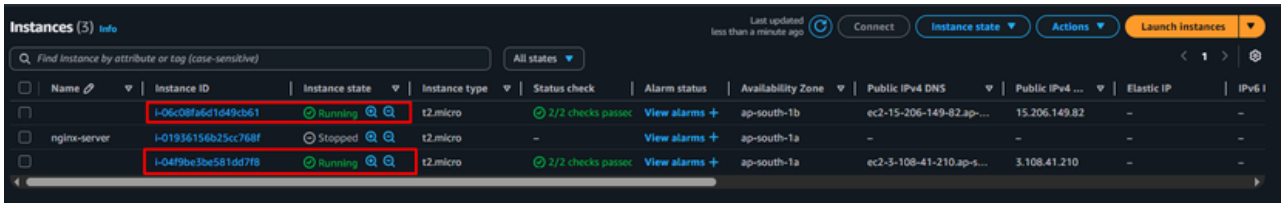


Hands-on Configure ASG:

Now, go to the Load Balancer section again and see what happened.



It shows two instances running, but one is unused, the one I stopped. Here are the instances:



Now browse and see if it works or not.



Yes! It works!

Thank You

Stay Connect:

/in/alamgirweb11

/alamgirweb11